



SENA
TECNÓLOGO ANÁLISIS Y DESARROLLO
DE SOFTWARE

TEMA: GA1-220501093-AA1-EV01- Taller. Metodologías de desarrollo de software.

AUTOR:

- **Gerson Jonnathan López Oviedo - 3186647**

DOCENTE:

- **Arnaldo Alfonso Montiel Brun**

BOGOTÁ D.C.

01 de julio del 2025

INTRODUCCIÓN

El desarrollo de software es una actividad compleja que requiere planificación, orden y trabajo en equipo para alcanzar productos funcionales y de calidad. Para lograrlo, existen las metodologías de desarrollo de software, que funcionan como una guía estructurada que orienta a los equipos desde la definición de las necesidades del cliente hasta la entrega y mantenimiento del producto final. Estas metodologías establecen procesos, roles, herramientas y buenas prácticas que permiten reducir errores, optimizar los recursos y garantizar que el software cumpla su propósito.

Dependiendo de las características del proyecto, se pueden utilizar marcos de trabajo tradicionales, que siguen un proceso lineal y estructurado, o marcos de trabajo ágiles, que permiten adaptarse rápidamente a los cambios y entregan resultados de manera progresiva. Ambos enfoques tienen ventajas y limitaciones, por lo que su correcta aplicación depende del contexto, los objetivos y el nivel de incertidumbre del proyecto.

A continuación, se presenta una descripción detallada de los principales conceptos, componentes y ejemplos de uso de estas metodologías, fundamentales para cualquier proceso de desarrollo de software en entornos educativos, empresariales o de formación profesional.

1. CONCEPTO METODOLOGIAS Y DESARROLLO DE SOFTWARE

Son como la “hoja de ruta” que sigue un equipo para crear sistemas o aplicaciones de manera ordenada y efectiva. Siempre parten desde un marco teórico, con un conjunto de pasos, reglas y buenas prácticas que ayudan a que el trabajo fluya bien: desde entender lo que necesita el cliente, hasta programar, aprobar y entregar el producto final. Usar una buena metodología no es solo seguir pasos, es tener un plan, evitar caos y entregar software que de verdad funcione y solucione problemas. Así el equipo no se enreda y el cliente queda contento.

Existen metodologías más **tradicionales**, como el modelo en cascada, que siguen un camino paso a paso; y otras más **ágiles**, como Scrum o XP, que permiten adaptarse rápido a los cambios, trabajar en equipo y entregar avances más frecuentes.

Estas metodologías son clave porque hacen más fácil organizar el trabajo, mejorar la calidad del software y asegurar que el producto final realmente sirva para lo que fue pensado.

2. COMPOSICIÓN DE LA METODOLOGÍA DE DESARROLLO DE SOFTWARE

se compone básicamente de **5 elementos clave** que ayudan a organizar, planear y ejecutar un proyecto de forma ordenada:

- **Procesos definidos:** pasos o etapas claras (como análisis, diseño, desarrollo, pruebas y mantenimiento).
- **Roles y responsabilidades:** cada miembro del equipo sabe qué debe hacer y cuándo.
- **Herramientas y técnicas:** se usan herramientas específicas (como Git, Jira, o diagramas UML) para apoyar el trabajo.
- **Buenas prácticas:** se aplican normas y patrones que ya han funcionado en otros proyectos.

- **Criterios de evaluación:** se revisa si lo que se desarrolla cumple con lo que se pidió y funciona bien.

3. EJEMPLOS DE UTILIDAD DE LA DE LA METODOLOGÍA DE DESARROLLO DE SOFTWARE:

- ✓ En un equipo grande, una metodología ayuda a que cada persona sepa qué hacer, evitando retrasos o trabajo duplicado.
- ✓ En proyectos complejos, permiten planear bien el tiempo y los recursos, logrando entregar el software a tiempo y con buena calidad.
- ✓ Cuando un cliente cambia de idea a mitad del proyecto, una metodología ágil como *Scrum* permite adaptarse rápido sin perder el rumbo ni romper todo el trabajo anterior.
- ✓ En proyectos educativos o de formación, aplicar una metodología facilita que los aprendices trabajen en equipo y sigan un proceso ordenado, como si fuera un simulacro del mundo laboral.
- ✓ En una empresa que desarrolla apps para clientes distintos, seguir una metodología garantiza que todos los proyectos mantengan un nivel de calidad estable sin importar quién los haga.

4. CARACTERÍSTICAS FUNDAMENTALES DE UN MARCO DE TRABAJO

TRADICIONAL:

Los marcos de trabajo tradicionales, como el modelo en cascada o RUP, se enfocan en **planear todo desde el principio**. Primero se recopilan los requisitos, luego se diseña, se desarrolla, se aprueba y al final se entrega el producto. Cada fase va en orden y no se pasa a la siguiente sin haber terminado bien la anterior.

Se caracterizan por tener:

- Mucha documentación desde el comienzo.
- Procesos bien organizados y formales.
- Cambios difíciles una vez que se arranca.
- Son útiles cuando el proyecto es estable, claro y no va a cambiar mucho.

En conclusión, el marco tradicional es como construir una casa con planos fijos desde el comienzo.

4.1 Tenemos los siguientes métodos tradicionales:

4.1.2 Modelo en cascada

- Es **secuencial**: se divide en fases que se hacen una tras otra (requerimientos → diseño → implementación → pruebas → mantenimiento).
- No se puede avanzar a la siguiente fase sin haber terminado bien la anterior.
- Es ideal para proyectos donde y no se espera que haya muchos cambios.
- Muy usado en proyectos cortos o con bajo nivel de incertidumbre.
- Ejemplo sencillo: Es como construir una casa con planos listos desde el principio. Si ya estás haciendo el techo, no puedes cambiar el diseño del piso.

4.1.3 RUP (Rational Unified Process)

- También es tradicional, pero un poco más **flexible que el método cascada**.

Se divide en **4 fases grandes**: Inicio, Elaboración, Construcción y Transición.

- Cada fase tiene **iteraciones internas**, lo que permite ajustar algunas cosas a mitad de camino.
- Se apoya mucho en los **diagramas, documentos y modelos**, y es ideal para proyectos grandes que requieren estructura.

Tabla comparativa de marcos de trabajo tradicionales			
Marco Tradicional	Características Clave	Ventajas	Ejemplo Sencillo
Cascada	- Fases secuenciales: primero una, luego la otra.	- Ideal para proyectos claros desde el inicio.	Hacer una casa: planos → cimientos → estructura → acabados.
	- Mucha planificación y documentación.	- Fácil de organizar.	
RUP (Proceso Racional Unificado)	- Fases definidas: inicio, elaboración, construcción, transición.	- Buen soporte técnico.	Desarrollar una app corporativa con roles claros y planificación fuerte.
	- Se basa en casos de uso.	- Reutilización de componentes.	
		- Bien documentado.	

5. CARACTERÍSTICAS FUNDAMENTALES DE UN MARCOS DE TRABAJO

ÁGILES

Los marcos ágiles, como Scrum o XP, son más **flexibles y colaborativos**. No se planea todo desde el comienzo, sino que se trabaja por partes (llamadas iteraciones), donde cada parte se revisa, se mejora y se adapta con base en el feedback del cliente o usuario.

Se caracterizan por:

- Adaptarse fácilmente a cambios durante el proceso.
- Mucha comunicación constante con el cliente y entre el equipo.
- Se enfoca en entregar valor rápido y frecuente.
- Menos documentos, más trabajo funcional.

En conclusión, el marco El ágil es como armar algo por partes, mejorándolo en cada vuelta según lo que se necesita o se aprende.

5.1. De la categoría de marcos de trabajo marcos de trabajo ágiles tenemos:

5.1.2. Scrum

- Es el más popular de los marcos ágiles.
- Se trabaja en ciclos cortos llamados sprints (normalmente de 2 semanas).
- El equipo se reúne todos los días (daily meeting) para revisar avances y bloqueos.
- Toda gira alrededor de entregas rápidas y funcionales, con mejora constante.

Hay tres roles clave: Product Owner, Scrum Master y el Equipo de desarrollo.

Ejemplo sencillo: Es como hacer un videojuego por partes. Primero haces que el personaje camine, luego que salte, luego que dispare... y cada dos semanas muestras lo nuevo y mejoras con base en el feedback.

5.1.3 Extreme Programming (XP)

- Se enfoca más en la calidad del código y la colaboración cercana con el cliente.
- Usa técnicas como programación en pareja, pruebas constantes, e integración intima.
- Está pensado para ambientes donde los requisitos cambian muy rápido.
- Da prioridad a la simplicidad y a una comunicación fluida dentro del equipo.

Ejemplo sencillo: Es como si dos programadores trabajaran en un mismo computador, uno escribe mientras el otro revisa, y prueban lo que hacen todo el tiempo. Así evitan errores grandes y se adaptan rápido.

5.1.4 3. Kanban

- Más que una metodología, es un sistema visual de trabajo.
- Se usan tableros para ver claramente qué tareas están por hacer, en proceso y terminadas.
- No trabaja en ciclos fijos, sino que las tareas fluyen según la capacidad del equipo.
- Se busca mejorar continuamente el proceso y limitar el trabajo en curso.

Ejemplo sencillo: Es como una cartelera con post-its donde ves claramente lo que estás haciendo, lo que ya terminaste y lo que viene. Súper visual y ordenado.

Tabla comparativa de marcos de trabajo ágiles			
Marco Ágil	Características Clave	Ventajas	Ejemplo Sencillo
Scrum	- Ciclos cortos llamados <i>sprints</i> .	- Entregas rápidas.	Desarrollar un juego por etapas cada 2 semanas.
	- Reuniones diarias.	- Mejora continua.	
	- Roles definidos.	- Buen control.	
Extreme Programming (XP)	- Programación en pareja.	- Alta calidad del código.	Dos devs (programadores) en un mismo PC probando cada cambio.
	- Pruebas constantes.	- Adaptación rápida.	
	- Alta comunicación.		
Kanban	- Tablero visual con tareas.	- Organización visual.	Usar post-its (papelito adhesivo) para saber qué se hace y qué sigue.
	- Flujo continuo.	- Trabajo flexible.	
	- Enfoque en limitar tareas abiertas.	- Mejora continua.	